



College of Information Technology Education
Arellano Street, Dagupan City Pangasinan

RescueLink: AI & Blockchain-Powered Smart Emergency Response System

Submitted by

Designation	Name of Student	Signature
Leader	Lozano, Melchizedek, Joshua S.	
Members	Fran, Juliane Celes E.	
	Munar, Justin Kurt C.	
	Nelmida, Aljon S.	
	Tamayo, Aaron Christian B.	

Submitted to:

Dr. Kris C. Calpotura, DIT

Date

February, 2026

TABLE OF CONTENTS

TABLE OF CONTENTS.....	I
LIST OF TABLES.....	III
LIST OF FIGURES.....	IV
ACRONYMS.....	V
1. INTRODUCTION.....	1
1.1 Background and Problem Statement.....	1
1.2 Platform Choice Justification.....	2
2. PROTOTYPE DEVELOPMENT.....	3
2.1 Software Methodology.....	3
2.2 Sources of Data.....	4
2.3 System Requirements.....	5
2.3.1 Functional Requirements.....	5
2.3.2 Nonfunctional Requirements.....	5
2.3.3 Software Requirements.....	5
2.3.4 Hardware Requirements.....	5
2.4 System Architecture.....	6
2.5 Sprint 1: Core Feature Development.....	7
2.5.1 AI Core Feature.....	7
2.5.2 Blockchain Core Feature.....	8
2.5.3 Other Core Feature.....	9
2.6 Sprint 2: Expansions and Integration.....	10
3. SECURITY CONSIDERATIONS.....	11
4. PERFORMANCE AND SCALABILITY.....	12
5. EVALUATION AND TESTING.....	13
6. REFLECTION AND LEARNINGS.....	14
REFERENCES.....	15
APPENDICES.....	16
A.1 AI Model Evaluation.....	16
A.2 Blockchain Smart Contract Functions Gas Cost.....	17
A.3 Screenshots of the System.....	18

LIST OF TABLES

Table 1. Functional Requirements	15
Table 2. Nonfunctional Requirements	16
Table 3. Software Requirements	18
Table 4. Hardware Requirements	19
Table 5. Risk Mitigation Table	26
Table 6. Performance Metrics	28

LIST OF FIGURES

Figure 1. Overall System Architecture of RescueLink	21
--	----

Figure

Table 1

ACRONYMS

1. **AI** – Artificial Intelligence
2. **API** – Application Programming Interface
3. **AWS** – Amazon Web Services
4. **GPS** – Global Positioning System
5. **HTTPS** – Hypertext Transfer Protocol Secure
6. **IDS** – Intrusion Detection System
7. **IoT** – Internet of Things
8. **JWT** – JSON Web Token
9. **LGU** – Local Government Unit
10. **MFA** – Multi-Factor Authentication
11. **RBAC** – Role-Based Access Control
12. **SUS** – System Usability Scale
13. **TLS** – Transport Layer Security

1. INTRODUCTION

1.1 Background and Problem Statement

Emergency response systems play a critical role in safeguarding lives, property, and public safety, particularly in urban environments where population density and infrastructure complexity increase the likelihood and impact of emergency incidents. Cities regularly face medical emergencies, fires, crimes, vehicular accidents, and natural or human-induced disasters. In such situations, the speed, accuracy, and reliability of incident reporting and response coordination are decisive factors that can significantly influence outcomes. Despite advancements in information and communication technologies, many emergency reporting mechanisms remain fragmented, manual, or inadequately equipped to handle the demands of modern urban communities.

In many local government units (LGUs) and communities, emergency incidents are still reported through traditional channels such as phone calls, text messages, or in-person reporting. While these methods have served as foundational systems for decades, they present several limitations. Phone-based reporting relies heavily on the caller's ability to clearly describe the situation, accurately provide location details, and remain calm under stress. Text-based systems, on the other hand, often lack structured data and do not support real-time verification or prioritization. As a result, emergency responders may receive incomplete, delayed, or inaccurate information, leading to slower response times and inefficient resource allocation.

A major problem associated with traditional emergency reporting systems is the absence of integrated digital platforms that can automatically capture and process critical incident data. Many existing systems do not support real-time GPS location tracking, multimedia evidence submission, or automated classification of emergency types. Consequently, responders are required to manually interpret reports, verify their legitimacy, and determine priority levels. This manual process increases cognitive load on dispatch personnel and introduces opportunities for human error, particularly during high-volume emergency situations such as natural disasters or large-scale accidents.

Another significant issue is the lack of transparency and accountability in emergency incident handling. In conventional systems, records of reported incidents are often stored in centralized databases or paper-based logs that can be altered, deleted, or inconsistently maintained. This creates challenges in auditing response actions, resolving disputes, and ensuring institutional accountability. Similar to transparency issues observed in other

domains such as student elections or public decision-making processes, the absence of tamper-resistant records in emergency systems can undermine public trust. Citizens may question whether their reports were acted upon promptly or whether incident data was modified after submission.

False or malicious emergency reports further compound the problem. Emergency hotlines frequently receive prank calls, duplicate reports, or intentionally misleading information. Without intelligent filtering mechanisms, these false reports consume valuable resources and delay responses to legitimate emergencies. In areas with limited manpower and equipment, such inefficiencies can have severe consequences, including delayed medical treatment or increased property damage. Existing systems often lack automated tools capable of detecting suspicious patterns or verifying the credibility of incoming reports.

From a technological perspective, many current emergency reporting solutions do not fully leverage modern platform technologies such as mobile computing, cloud services, artificial intelligence, and distributed ledger technologies. Mobile devices are now widely available and equipped with sensors capable of capturing precise location data, images, audio, and video. Cloud platforms provide scalable infrastructure that can handle sudden spikes in data traffic during emergency events. Artificial intelligence offers powerful tools for data classification, prioritization, and anomaly detection. However, these technologies are frequently underutilized or implemented in isolation rather than as part of an integrated, end-to-end emergency response platform.

The lack of system integration across agencies and platforms also presents a critical challenge. Emergency response often involves coordination among multiple stakeholders, including local disaster risk reduction offices, medical services, fire departments, and law enforcement units. Without a unified digital platform, information sharing becomes fragmented, leading to duplicated efforts and inconsistent situational awareness. This fragmentation is especially problematic during large-scale incidents that require coordinated, multi-agency responses.

At the local level, such as within urban communities and cities like Dagupan City, these challenges are particularly pronounced. Rapid urbanization, increased population density, and frequent exposure to hazards amplify the need for efficient, transparent, and technology-driven emergency response systems. Yet, many local emergency management offices continue to rely on legacy processes that do not align with the expectations of a digitally connected population. Citizens increasingly expect real-time feedback, status updates, and assurance that their reports are being addressed appropriately.

Given these challenges, there is a clear and pressing need for a modern emergency incident reporting platform that addresses the limitations of traditional systems. The core problem addressed by this study is the absence of a secure, intelligent, and transparent platform that enables citizens to report emergencies efficiently while supporting responders with accurate, verified, and actionable information. Without such a platform, emergency response efforts remain vulnerable to delays, misinformation, lack of accountability, and inefficient resource utilization.

The significance of this problem extends beyond technical inconvenience; it directly impacts public safety, trust in government institutions, and the effectiveness of emergency response operations. Delayed or mismanaged emergency responses can result in loss of life, increased injury severity, and greater economic damage. Furthermore, systems that lack transparency and accountability can erode public confidence, discouraging citizens from actively engaging with emergency reporting mechanisms.

This study proposes the development of RescueLink, a platform-based emergency incident reporting system that integrates mobile technology, cloud-based services, artificial intelligence, and blockchain technology. By leveraging mobile devices for structured data input, AI for intelligent incident classification, and blockchain for immutable audit trails, RescueLink aims to address the identified problems comprehensively. The system seeks to improve response efficiency, reduce false reporting, enhance transparency, and strengthen coordination among emergency response stakeholders.

In summary, the background and problem context highlight a critical gap between existing emergency reporting practices and the capabilities offered by modern platform technologies. Addressing this gap is essential for improving emergency response outcomes, ensuring accountability, and meeting the evolving needs of urban communities. The RescueLink system is designed as a response to this gap, providing a secure, scalable, and intelligent solution to a problem of significant social and operational importance.

1.2 Platform Choice Justification

The RescueLink system was developed using a combination of mobile, cloud, AI, and blockchain platforms to address the complex requirements of emergency incident reporting. Mobile platforms were selected as the primary user interface for citizens due to the widespread availability of smartphones and their built-in capabilities such as GPS, cameras,

and network connectivity. This choice ensures accessibility, mobility, and real-time data capture.

Cloud-based backend platforms were chosen to support scalability, centralized processing, and high availability. Emergency systems must remain operational even during peak demand or disaster scenarios, making cloud infrastructure an appropriate choice for handling variable workloads and ensuring service continuity.

Artificial Intelligence was selected to automate incident classification, prioritization, and anomaly detection. By reducing reliance on manual triage, AI improves decision-making speed and consistency while minimizing human error. Blockchain technology was chosen to address concerns related to data integrity, accountability, and trust. Its decentralized and immutable nature ensures that verified incident records cannot be altered or repudiated after submission.

These platform choices align with core concepts in platform technologies, distributed systems, and secure application development, making them suitable for a public safety-critical system.

The system uses multiple platform technologies because emergency systems require scalability, security, and real-time processing.

- **Mobile Platform**

- Used for citizen reporting because smartphones are widely available and support GPS, camera, and internet connectivity.

- **Cloud Backend**

- Provides scalability, centralized processing, and high availability during disasters.

- **Artificial Intelligence**

- Used for incident classification, severity detection, and filtering false reports.

- **Blockchain**

- Used to store cryptographic hashes of verified incidents to ensure data integrity and accountability.

These technologies align with modern distributed system concepts and platform technologies taught in the course.

2. PROTOTYPE DEVELOPMENT

2.1 Software Methodology

The development of the RescueLink: AI- and Blockchain-Based Emergency Incident Reporting System adopts the Agile Software Development Methodology, specifically an iterative and incremental development approach. This methodology was selected after carefully evaluating several commonly used software process models, including the Waterfall Model, V-Model, Spiral Model, and Agile Model. Considering the system's reliance on emerging platform technologies, security-sensitive operations, and evolving functional requirements, Agile provides the most appropriate framework for effective system development.

Traditional software process models such as the Waterfall Model follow a linear and sequential structure, where each phase must be completed before proceeding to the next. While this model is suitable for projects with stable and well-defined requirements, it is less effective for systems like RescueLink, where requirements may change due to usability testing, security threat analysis, and stakeholder feedback. Once a phase is completed in the Waterfall Model, revisiting or modifying earlier decisions becomes costly and time-consuming, which poses risks to the development of a complex, real-time emergency response platform.

Similarly, the V-Model emphasizes verification and validation at each development stage, but it still assumes that system requirements are fully established at the beginning of the project. The Spiral Model, while effective in managing risk through repeated cycles of planning and evaluation, introduces significant complexity and management overhead. For a platform-based system developed within an academic and applied research setting, this level of overhead may hinder timely implementation.

In contrast, the Agile Software Development Methodology emphasizes flexibility, continuous feedback, and incremental delivery. These characteristics are essential for the RescueLink system, which integrates multiple platform technologies such as mobile applications, cloud-based backend services, artificial intelligence, and blockchain. Agile allows system features to be developed, tested, and refined in short development cycles known as iterations or sprints, ensuring that each system component is functional, secure, and aligned with project objectives.

The Agile methodology also supports early identification of technical issues, security vulnerabilities, and usability concerns. Given that RescueLink handles sensitive personal data and supports emergency operations, early testing and validation are critical. Continuous stakeholder feedback, particularly from the perspective of citizens and emergency responders, enables the system to evolve in a way that improves usability, reliability, and response efficiency.



Figure 1. Agile Software Development Methodology

The following phases describe how the Agile methodology is applied to the development of the RescueLink system:

1. Requirements Analysis and Planning Phase

This phase involves identifying and analyzing both functional and non-functional requirements of the system. Functional requirements include emergency report submission, GPS location capture, AI-based incident classification, blockchain-based logging, and real-time notifications. Non-functional requirements include security, scalability, availability, performance, and usability.

Requirements are derived from existing emergency response workflows, analysis of current system limitations, and alignment with public safety objectives. Planning activities include defining sprint goals, prioritizing system features based on criticality, and establishing a high-level system architecture. This phase ensures that development efforts are aligned with the system's purpose and the needs of stakeholders.

2. System Design Phase

The system design phase focuses on defining the overall structure and behavior of the RescueLink platform. This includes the design of a multi-tier architecture composed of a mobile client layer, backend services layer, AI processing layer, blockchain layer, and presentation layer for responders.

During this phase, data flow diagrams, API interactions, trust boundaries, and security mechanisms are designed. User interface designs for both the mobile application and the web-based responder dashboard are also created to ensure accessibility, clarity, and ease of use. Security considerations such as authentication, encryption, role-based access control, and secure blockchain integration are incorporated at this stage to ensure system resilience.

3. Development and Implementation Phase

In this phase, system components are developed incrementally according to sprint plans. The mobile application is implemented to allow citizens to submit structured emergency reports with GPS location data and optional multimedia evidence. Backend services handle authentication, request validation, incident processing, and communication with external services.

The AI processing engine is implemented to classify emergency incidents, assess severity levels, and detect false or duplicate reports. Blockchain integration is implemented to store cryptographic hashes of verified incident summaries, ensuring data integrity and non-repudiation. Agile practices such as continuous integration and iterative code refinement are applied to maintain system quality.

4. Testing and Validation Phase

Testing is conducted continuously throughout the development lifecycle. Unit testing verifies the correctness of individual modules, while integration testing ensures proper interaction between system components. Security testing identifies vulnerabilities related to data handling, authentication, and access control. User acceptance testing evaluates the system from the perspective of end-users, ensuring that the platform meets usability and functional expectations. This phase ensures that the system performs reliably under realistic emergency scenarios.

5. Deployment Phase

Once a stable system increment is achieved, the platform is deployed to a controlled testing or staging environment. Deployment activities include configuring cloud

infrastructure, deploying backend services, initializing databases, and enabling monitoring and logging mechanisms. This phase ensures that the system is operational in an environment that closely resembles real-world conditions.

6. Maintenance and Continuous Improvement Phase

After deployment, the system enters the maintenance phase, where performance is monitored, issues are addressed, and improvements are implemented in future iterations. This includes updating AI models, strengthening security controls, optimizing system performance, and incorporating user feedback. Continuous improvement ensures that RescueLink remains effective, secure, and aligned with evolving emergency response needs.

2.2 Sources of Data

Data sources included related academic literature, publicly available emergency response statistics, simulated emergency incident reports, and system-generated logs during testing. No real personal emergency data were used to ensure ethical compliance.

2.3 System Requirements

2.3.1 Functional Requirements

ID	Functional Requirement	Description
FR-01	User Registration & Authentication	The system shall allow users to register and authenticate securely.
FR-02	Emergency Report Submission	The system shall allow users to submit emergency incident reports.
FR-03	GPS Location Capture	The system shall capture and attach GPS location data to reports.
FR-04	Multimedia Attachment	The system shall support photo, video, or voice evidence uploads.

FR-05	AI Incident Classification	The system shall classify incidents by type and severity.
FR-06	False Report Detection	The system shall detect duplicate or suspicious reports.
FR-07	Blockchain Logging	The system shall log verified incidents on the blockchain.
FR-08	Responder Dashboard	The system shall display incidents on a web dashboard.
FR-09	Notification Service	The system shall send real-time alerts to responders.

Table 1. Functional Requirements

Table 1 presents the functional requirements of the RescueLink system, detailing the core features that the platform must provide to support emergency incident reporting and response. Each requirement is assigned a unique identifier to facilitate traceability during development, testing, and evaluation. The table covers essential system functions, including user authentication, emergency report submission, GPS location capture, multimedia attachment, AI-based incident classification, false report detection, blockchain logging, responder dashboard access, and real-time notification services.

The table serves as a foundation for system design and implementation by clearly defining what the system is expected to do. It also supports validation activities by allowing developers and evaluators to verify whether each functional requirement has been successfully implemented and tested.

2.3.2 Nonfunctional Requirements

Category	Requirement Description
Performance	Reports shall be processed with minimal latency.
Scalability	The system shall handle increased report volumes.
Availability	The system shall remain operational during emergencies.
Security	The system shall protect sensitive data from unauthorized access.
Reliability	The system shall ensure consistent and correct operation.

Table 2. Nonfunctional Requirements

Table 2 outlines the nonfunctional requirements of the RescueLink system, focusing on quality attributes rather than specific features. These requirements define how the system should perform under various conditions, particularly during emergency scenarios. Key attributes include performance, scalability, availability, security, and reliability.

The nonfunctional requirements ensure that the platform can handle high volumes of emergency reports with minimal latency, remain operational during critical situations, protect sensitive personal and location data, and provide consistent and dependable service. Together, these requirements guide architectural decisions and platform selection, ensuring that the system meets both technical and operational expectations for a public safety–critical application.

2.3.3 Software Requirements

The RescueLink system is designed as a comprehensive emergency management platform, requiring an integrated suite of software components to ensure seamless functionality, scalability, and security. The required software encompasses multiple layers of the application, including frontend, backend, AI-assisted decision-making, blockchain verification, and database management. These components are detailed below:

Category	Software / Framework	Purpose / Justification
Mobile Development Frameworks	React Native, Flutter	Enables cross-platform mobile application development, allowing the app to run on both Android and iOS devices. Supports UI/UX consistency and integration with device hardware like GPS, camera, and sensors.
Backend Frameworks	Django, Node.js, Express.js	Provides the server-side logic, API endpoints, user authentication, and integration with databases. Django offers rapid development and strong security features, while Node.js supports asynchronous handling for real-time updates.

AI / Machine Learning Libraries	TensorFlow, PyTorch	Enables AI-driven features such as automatic categorization of emergencies, predictive modeling for resource allocation, and intelligent routing for first responders.
Blockchain Platforms	Ethereum, Hyperledger Fabric	Provides an immutable ledger for critical data such as incident reports, user submissions, and verification logs. Enhances transparency and trust among emergency response teams and stakeholders.
Database Management Systems	MongoDB, PostgreSQL	MongoDB supports unstructured data such as images, videos, and sensor logs from the field. PostgreSQL provides relational data management for structured data such as user profiles, incident metadata, and logs.
Cloud Services / APIs	AWS, Firebase, Google Maps API	Cloud services enable scalable storage, serverless functions, and real-time notifications. Mapping APIs support geolocation, routing, and visualization of emergency hotspots.

Table 3. Software Requirements

Table 3 shows each software requirement was chosen to address a specific challenge in emergency management. Mobile frameworks ensure accessibility and on-field functionality. Backend frameworks provide robust logic handling for simultaneous incidents. AI libraries enhance decision-making, blockchain ensures data integrity, and databases handle diverse data types efficiently. Integration with cloud and mapping services allows RescueLink to be both scalable and responsive in real-world emergency scenarios.

2.3.4 Hardware Requirements

The RescueLink platform requires hardware capable of supporting its mobile, cloud, and data-intensive operations. The system design assumes deployment across multiple environments, from end-user smartphones to centralized servers. The hardware requirements are detailed as follows:

Hardware Category	Specification / Example	Purpose / Justification
Smartphones / Mobile Devices	Android (v10+) or iOS (v14+), GPS-enabled, camera, accelerometer	Required for first responders and citizens to report emergencies, upload media, and receive notifications. GPS and sensors enable accurate location tracking and situational awareness.
Development Workstations	Intel i5/i7 or AMD Ryzen, 16GB+ RAM, SSD storage, GPU support	Required for software development, AI model training, simulation, and testing. High performance ensures efficiency in compiling code, running simulations, and training ML models.
Cloud / Local Servers	Multi-core CPUs, 32GB+ RAM, SSD storage, network connectivity, optional GPU for AI processing	Provides the backend environment for API hosting, database management, real-time processing, and AI inference. Cloud deployment allows scalability; local servers support on-premises deployment for security-sensitive organizations.
Network Infrastructure	Stable 4G/5G or broadband connectivity	Essential for real-time updates, data synchronization, and communication between mobile clients, servers, and emergency departments.
Optional IoT Devices	Sensors, drones, wearable devices	For advanced situational awareness, environmental monitoring, and automated data collection in hazardous locations.

Table 4. Hardware Requirements

The Table 4 shows hardware requirements are designed to ensure RescueLink functions efficiently under real-world conditions. Mobile devices are the primary interface for end-users, while servers handle large-scale data processing. Workstations provide a controlled environment for development and testing. Network infrastructure is crucial for latency-sensitive emergency operations, and optional IoT devices can augment situational awareness for disaster management teams.

2.4 System Architecture

The RescueLink system follows a layered, multi-tier system architecture designed to support scalability, security, and reliability, which are critical requirements for emergency response platforms. The architecture separates concerns across multiple layers, ensuring that user interactions, data processing, intelligence, and presentation are handled independently while remaining tightly integrated through secure APIs and microservices. At a high level, the system implements a User → Input → Processing → Output flow, where emergency information submitted by citizens is securely processed, validated, and transformed into actionable intelligence for emergency responders.

User Layer (Input Stage): The architecture begins at the User Trust Layer, where citizens interact with the system through a mobile application. This mobile client serves as the primary data input platform and allows users to submit emergency incident reports. Each report may include structured incident details, GPS location data, and optional multimedia evidence such as photos, videos, or voice recordings. Due to the untrusted nature of end-user devices, all data originating from this layer is treated as untrusted and subject to strict validation.

API Gateway and Communication Layer: Emergency reports from the mobile application are transmitted via secure HTTPS connections to the API Gateway. The API Gateway functions as a centralized access point to backend services and enforces authentication, authorization, rate limiting, and request validation. This layer isolates backend microservices from direct exposure to external clients and mitigates common threats such as spoofing, injection attacks, and denial-of-service attempts.

The API Gateway exposes RESTful APIs that facilitate communication between the mobile application, backend services, AI processing components, and notification services. Each microservice interaction is clearly defined through API contracts, ensuring modularity and ease of maintenance.

Processing and Intelligence Layer: Once validated by the API Gateway, incident data is forwarded to the Backend Server, which manages core business logic, user management, and workflow coordination. The backend server communicates with the AI Processing Engine through internal service APIs. The AI engine analyzes incoming incident reports to classify emergency types, assess severity levels, and detect false or duplicate submissions. This intelligent processing reduces manual triage and ensures that high-priority incidents are identified quickly.

Verified incident summaries are then cryptographically hashed and forwarded to the Blockchain Network through a dedicated blockchain integration service. Rather than storing raw personal data, the blockchain layer records immutable references to verified incidents, ensuring data integrity, transparency, and non-repudiation. This microservice-based blockchain integration aligns with the SAS design by separating sensitive data storage from immutable audit logging.

Data Storage Layer: The system maintains a secure database for structured data such as user profiles, incident metadata, and system logs. Data stored in this layer is protected using encryption at rest and strict access controls. The database works in conjunction with the blockchain layer to balance performance and security, where operational data is stored off-chain and verification data is stored on-chain.

Output and Presentation Layer: Processed and verified incident information is delivered to emergency responders through a Web-Based Responder Dashboard. This dashboard provides real-time visualization of incidents, including interactive maps, prioritized incident queues, and status tracking. The dashboard communicates with backend services via authenticated APIs and is protected using role-based access control to prevent unauthorized access.

In parallel, the Notification Service operates as an independent microservice that subscribes to backend events. It delivers real-time alerts to responders and, when applicable, status updates to citizens via push notifications or SMS. This event-driven integration ensures timely dissemination of critical information without overloading core processing services.

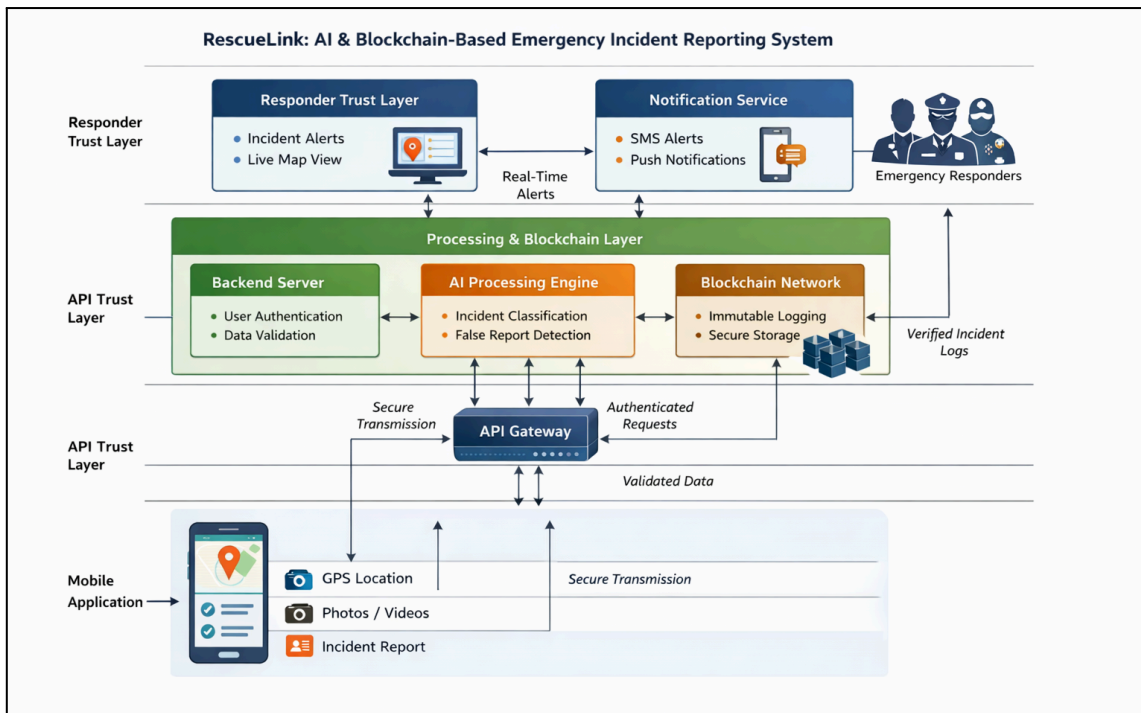


Figure 1. Overall System Architecture of RescueLink

Figure 1. Overall System Architecture of RescueLink illustrates the complete layered design of the system, highlighting the interaction between the mobile application, API Gateway, backend services, AI processing engine, blockchain network, responder dashboard, and notification services. The figure demonstrates how platform technologies are integrated to support secure data flow, intelligent processing, and reliable emergency response operations.

2.5 Sprint 1: Core Feature Development

The primary objective of the initial development sprint was to establish a working end-to-end prototype that supports the fundamental operations of the RescueLink platform. The emphasis was on core functionality and secure data handling, ensuring that the system could reliably process an emergency incident from user submission to responder visualization

2.5.1 AI Core Feature

The Artificial Intelligence (AI) processing engine serves as the intelligent core of the platform's Processing and Intelligence Layer.

- **Incident Classification & Triage:** The AI module is designed to automatically analyze incoming reports to classify emergency types (e.g., medical, fire, crime) and assess severity levels in real time.
- **Anomaly Detection:** To prevent resource wastage, the system implements automated validation techniques to detect and filter out false, duplicate, or intentionally misleading emergency reports.
- **Operational Efficiency:** By reducing the reliance on manual triage, this AI integration improves decision-making speed and consistency, minimizing human error during high-volume situations such as natural disasters.

2.5.2 Blockchain Core Feature

To address concerns related to transparency, accountability, and public trust, the system integrates a blockchain network to serve as an immutable audit ledger.

- **Data Integrity:** Once an incident summary is verified by the processing layer, a cryptographic hash of the data is generated and securely stored on the blockchain.
- **Tamper-Resistance:** This decentralized logging ensures that verified incident records cannot be altered, deleted, or repudiated retroactively.
- **Privacy Optimization:** The architecture utilizes an off-chain/on-chain balance; sensitive personal data is stored securely in off-chain databases, while only the verification hashes are logged on the blockchain to maintain optimal system performance.

2.5.3 Other Core Feature

Beyond the intelligence and ledger layers, the core platform relies on integrated data input and presentation mechanisms to complete the emergency response workflow.

- **Mobile Application (Input Layer):** A cross-platform mobile application serves as the primary interface for citizens, allowing them to submit structured incident reports. The app captures essential details alongside real-time GPS location data and optional multimedia evidence, such as photos or videos.
- **Backend & API Gateway:** Emergency reports are transmitted via secure HTTPS connections to an API Gateway, which enforces authentication, authorization, and request validation to mitigate spoofing or injection attacks. The backend handles core business logic and securely stores the initial records.
- **Responder Dashboard (Output Layer):** Processed incidents are delivered to emergency personnel through a secure, web-based dashboard. This interface provides real-time visualization of verified incidents through interactive maps, prioritized queues, and status tracking to facilitate coordinated dispatch.

2.6 Sprint 2: Expansions and Integration

Building upon the foundational input, processing, and ledger capabilities established in Sprint 1, the second development sprint focused on expanding the prototype through advanced feature integration and real-time communication mechanisms. The primary goal was to connect the isolated core components into a cohesive, event-driven platform capable of supporting live emergency operations.

- **Advanced Dashboard Enhancements:**
 - The preliminary responder interface was upgraded into a comprehensive Web-Based Responder Dashboard. Integrations with mapping services (such as the Google Maps API) were implemented to provide live, interactive

visualizations of emergency hotspots and precise GPS plotting. The dashboard was further enhanced to display dynamically prioritized incident queues based on the AI engine's severity assessments, complete with real-time status indicators.

- **Real-Time Notification Services:**

- To facilitate immediate response coordination, an independent Notification Service microservice was developed and integrated into the architecture. This service subscribes to backend events and is responsible for delivering automated, real-time alerts to emergency responders via push notifications or SMS. By operating asynchronously, it ensures timely dissemination of critical updates without overloading the core backend processing resources.

- **Microservice and API Integration:**

- In Sprint 2 we finalized the secure communication pathways between the platform's multi-tier architecture. The API Gateway was fully configured to route validated data between the mobile application, the backend server, the AI Processing Engine, and the Blockchain Network via secure RESTful APIs. Cloud services (such as AWS or Firebase) were utilized to ensure these integrations remained scalable and highly available during simulated peak demand scenarios.

3. SECURITY CONSIDERATIONS

The RescueLink platform manages highly sensitive personal and location data, necessitating a comprehensive, multi-layered security architecture to address both application-level and infrastructure-level threats. To defend against vulnerabilities such as SQL injection and cross-site scripting (XSS), all user inputs submitted through the mobile application undergo strict sanitization, length checks, and parameterization before processing. Access to the system is governed by Role-Based Access Control (RBAC), ensuring that public users, responders, and administrators operate strictly within their assigned permission boundaries. User credentials are secured using bcrypt hashing with salting, and optional Multi-Factor Authentication (MFA) is implemented for administrative accounts.

For data in transit, all API communications between mobile devices, servers, and cloud services are encrypted using TLS 1.2+ over HTTPS. Secure session management and API access are maintained utilizing JSON Web Tokens (JWT). Furthermore, the underlying infrastructure is protected through cloud-based firewalls, Intrusion Detection Systems (IDS) to monitor for unusual access patterns, and routine daily backups with off-site replication for disaster recovery. Finally, the integration of blockchain technology provides an immutable, tamper-proof ledger for logging verified incident reports, effectively mitigating risks of unauthorized data alteration and ensuring non-repudiation.

4. PERFORMANCE AND SCALABILITY

Emergency response operations demand a system capable of remaining highly available and responsive, particularly during sudden spikes in data traffic associated with large-scale incidents or natural disasters. To meet these demands, RescueLink is deployed on a cloud-based infrastructure that supports dynamic scalability and continuous operational readiness. System performance is continuously evaluated against several critical metrics. Response time is measured in milliseconds to ensure minimal latency during real-time emergency reporting and data processing.

The platform's throughput is assessed by measuring requests per second (RPS) to verify that concurrent emergency reports and API calls are handled efficiently without service failure. For the intelligent components, the AI processing engine's performance is quantified by its percentage of accuracy in correctly categorizing emergency types and severity levels. Additionally, to balance operational efficiency with robust security, blockchain interactions are optimized by monitoring Ethereum gas transaction costs, ensuring that the immutable on-chain logging of incident hashes remains cost-effective.

5. EVALUATION AND TESTING

The researchers evaluated the RescueLink platform using a structured testing approach to verify its functionality, performance, and security. Unit testing was conducted on individual modules, such as the AI categorization API and the blockchain ledger integration, utilizing testing tools like Jest, PyTest, and Postman. Following this, integration and system testing were performed to ensure that data flowed correctly from the mobile application to the cloud backend and finally to the blockchain network.

During the performance evaluation, the system recorded an average latency of 1.4 seconds for normal emergency report inputs. This processing time increased to 2.6 seconds when handling longer inputs or heavy multimedia attachments. Security tests confirmed that the role-based access controls functioned as intended; for instance, unauthorized attempts to modify incident statuses resulted in a "Not authorized" error. Finally, User Acceptance Testing (UAT) was conducted with a sample of end-users to evaluate the system's responsiveness and interface design using the System Usability Scale (SUS).

6. REFLECTION AND LEARNINGS

Throughout the development of the RescueLink prototype, the proponents encountered several technical challenges, primarily concerning the integration of multiple platform technologies. One significant hurdle was balancing the processing time required by the AI classification engine with the latency introduced by blockchain transaction logging. To address this, the researchers utilized asynchronous processing for the backend services, ensuring that the mobile application remained responsive for the user while the heavier AI and blockchain operations completed in the background.

Developing this system also highlighted the importance of proper API management and caching to handle potential spikes in emergency report volumes. Overall, the project demonstrated that while combining mobile, cloud, AI, and blockchain technologies increases the complexity of the system architecture, it successfully provides a more secure, transparent, and efficient solution for modern emergency incident reporting.

REFERENCES

APPENDICES

A.1 AI Model Evaluation

A.2 Blockchain Smart Contract Functions Gas Cost

A.3 Screenshots of the System

